포트폴리오

Proficiency

- Spring Boot & Java 기반으로 안정적인 백엔드 애플리케이션을 설계하고 구현할 수 있습니다.
- 데이터 조회 구조와 SQL을 분석하여 N+1, 비효율적인 집계 쿼리, 정렬 병목과 같은 성능 문제를 개선할 수 있습니다.
- 서비스 지표와 로그를 기반으로 문제를 재현 → 분석 → 개선 → 검증하는 방식으로 접근합니다.
- 트래픽이 집중되는 구간에서 캐시와 배치 반영 구조를 활용해 동시성 및 쓰기 병목을 완화한 경험이 있습니다.
- 새로운 기술을 적용할 때 동작 원리뿐 아니라 운영 복잡도와 비용까지 함께 확인하려고 합니다.

Project Portfolio

TeenPle | 학교 인증 기반 고등학생 익명 커뮤니티

2025.10 - 2026.07 개발 • 출시 이후 운영 • 2인 팀 • Backend ・ iOS / Android 실제 출시

학생증 기반 학교 인증을 완료한 사용자가 익명 게시판, 댓글, 실시간 채팅, 알림, 시간표, 신고•제재 기능을 이용하는 모바일 커뮤니티 서비스입니다. 전체 ERD와 주요 도메인 API를 설계•구현했고, 개발 과정에서 발견한 조회•검색•동시성 병목을 k6, p6spy, EXPLAIN ANALYZE로 재현하고 개선했습니다.

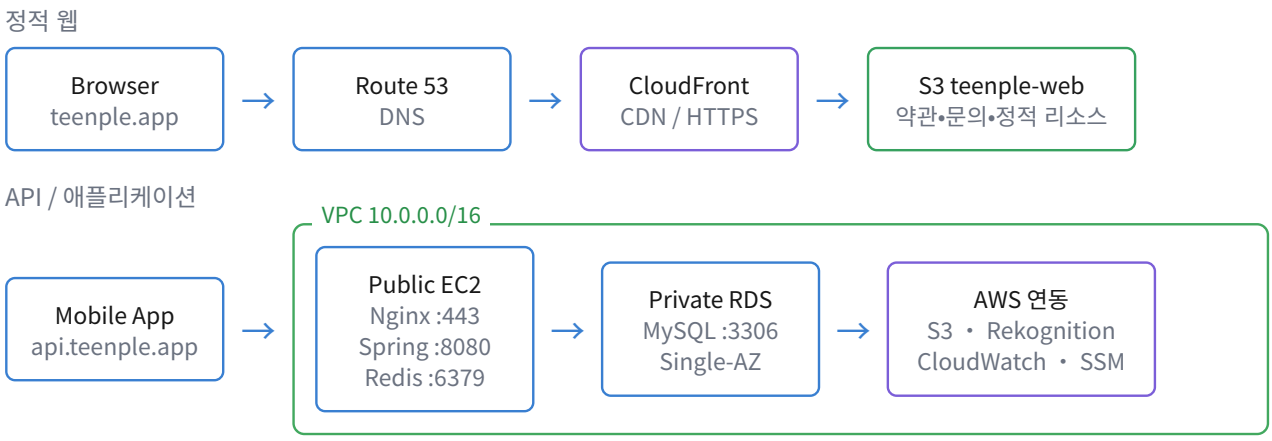
주요 담당

- 전체 ERD 및 게시글·댓글·알림·채팅·신고·제재·시간표 도메인 설계
- REST API, JWT/Spring Security 인증·인가
- WebSocket 채팅, Firebase Push 연동
- AI 활용 디자인 설계 및 Flutter(Dart) 앱 개발
- 성능 테스트와 SQL 분석을 통한 조회·쓰기 병목 개선

핵심 기술

- Java · Spring Boot · Spring Data JPA
- MySQL · Redis · WebSocket · Firebase
- Flutter · Dart
- AWS EC2 · RDS · S3 · Route 53 · CloudFront · CloudWatch

현재 운영 아키텍처



Hot 게시글 조회 API - 조회수 동시성 및 쓰기 병목 개선

문제 상황

게시글 상세 조회마다 조회수 DB UPDATE 발생. 동일 postId에 요청이 몰리는 Hot 게시글에서 하나의 Row에 쓰기 경합 집중, 동시성 제어 방식에 따라 지연 급증•실패율 증가.

비교 기준

동일 postId 집중 트래픽을 k6 시나리오(VU 최대 200)로 고정하고 성공률•처리량•p95를 함께 비교. 정합성뿐 아니라 실제 요청 처리 성능까지 동일 조건에서 검증.

시도	실험에서 확인한 점	결과
DB Atomic Update	lost update는 제거됐지만 요청마다 같은 Row를 UPDATE해 DB write 경합이 계속 발생	정합성은 해결, p95 병목은 남음
Pessimistic Lock	정합성은 보장됐지만 Hot Row의 lock 대기 시간이 증가	처리량이 더 악화
Optimistic Lock + retry	동일 Row 충돌이 빈번해 retry가 반복됨	성공률과 지연이 크게 악화
Caffeine Write-Behind	요청 시 메모리 카운터를 증가시키고 누적값을 주기적으로 DB에 반영	요청 경로의 DB write 제거

Write-Behind로 전환

Atomic Update와 비관적•낙관적 락 모두 요청마다 DB UPDATE가 발생하는 구조는 동일. 조회 요청에서는 DB 직접 갱신 대신 Caffeine 카운터에 누적하고 Batch로 반영하는 Write-Behind 방식으로 전환.

Flush 주기

Flush 주기 10초•30초•60초 비교. 짧은 주기는 DB 반영 빈도 증가, 긴 주기는 장애 시 미반영 범위와 사용자 체감 반영 지연 증가. 성능과 반영 지연의 균형을 고려해 30초를 기본값으로 적용.

64 → 453~483 req/s
처리량 약 6.0~7.6배 증가

2.27s → 0.45~0.59s
p95 약 74~80% 개선

30s Flush
성능과 반영 지연을 비교해 선택

> **결과** 락 방식 변경보다 조회 요청 경로의 DB UPDATE 자체를 제거했을 때 처리량 및 tail latency 개선 폭이 가장 큼.

상세 정리: ① InnoDB Row Lock vs JPA 비관적/낙관적 락 ② Caffeine Write-Behind 적용

게시글 검색 API - LIKE → FullText → OpenSearch 단계적 검증

1) LIKE 검색에서 병목 위치 분리

`title LIKE '%keyword%' OR content LIKE '%keyword%'` 형태의 검색에서 post 20,000건 중 약 6,000건이 매칭되는 키워드 조회. 필요한 결과는 최신순 21건이지만 대량 후보 생성 후 정렬•limit을 수행해 약 200ms 지연 발생.

검색을 Phase1(조건에 맞는 postId를 size+1만 조회)과 Phase2(postId로 게시글•작성자•댓글 수 상세 조립)로 분리 측정. Phase2는 0.1ms 이하로 미미했고, 대부분의 비용이 후보 생성과 최신순 정렬에 집중됨.

방식	측정	확인한 한계
LIKE	Phase1 약 200ms Phase2 0.1ms 이하	부분 문자열 검색으로 후보 생성•정렬 비용이 큼
MySQL FullText	Phase1 약 138ms Phase2 0.08~0.14ms	후보 탐색은 개선됐지만 학교/지역 필터와 최신순 정렬 비용이 남음
OpenSearch PoC	엔진 내부 took 11~30ms	MySQL 수치와 측정 경계가 달라 end-to-end 배수 비교에는 사용하지 않음

2) MySQL FullText 적용

FullText 적용 후 키워드 후보 탐색 비용 감소. 다만 후보가 많은 키워드에서는 최신순 정렬과 게시판 접근 범위(학교/지역) 필터 비용이 계속 남음. Phase1/Phase2 구조는 유지해 상세 조립 비용 증가 방지.

3) OpenSearch PoC

게시글 20,000건을 별도 역인덱싱하고 OpenSearch에서 id를 검색한 뒤 MySQL에서 상세를 조립하는 구조 검증. curl로 동일 키워드를 반복 호출하며 엔진 내부 처리시간 `took`과 네트워크•클라이언트 오버헤드가 포함된 total time을 분리 확인.

운영에는 바로 넣지 않은 이유

OpenSearch의 후보 탐색 성능 가능성 확인. 다만 MySQL 데이터 동기화, 인덱스 매핑•분석기 관리, 검색 엔진 장애 시 fallback, 모니터링•비용 부담 추가. 현재 서비스 규모에서는 운영 도입 대신 PoC 상태 유지.

상세 정리: 검색 API 성능 고찰 (1) • (2) OpenSearch란 • (3) 도입 검증

게시글 목록 조회 - N+1로 증가하던 DB Round Trip 제거

문제 상황

게시글 목록 API에서 게시글 N개 조회 후 각 게시글의 댓글 수를 위해 `countByPostId()`를 N번 호출. DTO 변환 과정의 LAZY 연관 엔티티 추가 조회 가능성까지 있어 목록 1회 호출이 게시글 수에 비례해 DB 호출을 늘리는 구조.

쿼리 수부터 확인

p6spy로 API 1회 호출의 실제 SQL 추적, 총 23회 Query 발생 확인. 특정 COUNT SQL 하나의 실행시간보다 요청 한 번에서 반복되는 DB Round Trip을 우선 개선 대상으로 두고 조회 구조 변경.

Fetch Join 대신 2-Phase를 선택한 이유

연관 데이터를 한 번에 가져오는 Fetch Join도 검토. 그러나 목록 페이징에서 Collection Fetch Join 사용 시 Join 결과 Row 증가로 중복•페이지 정확성 문제가 발생할 수 있음. 페이지 대상 ID를 먼저 고정한 뒤 상세를 조립하는 2-Phase 방식으로 분리.

단계	처리	목적
Phase 1	페이지에 필요한 postId를 size+1만 조회	페이징 대상 고정
Phase 2	postId 집합으로 게시글 상세 + 댓글 수를 한 번에 조립	댓글 수는 서브쿼리 집계/집계 조인으로 처리

23 → 2
API 1회 Query 수 약 91% 감소

28~53 → 16~18ms
평균 응답시간 약 35~60% 개선

60 → 22~27ms
p95 50% 이상 개선

> **다음 단계** DB Round Trip 우선 축소 후, 목록에서 반복되는 댓글 COUNT 쿼리는 실행 계획과 인덱스를 기준으로 추가 최적화.

댓글 COUNT 쿼리 - 실제 조회 조건에 맞춘 복합 인덱스 적용

문제 상황

댓글 수 조회에서 `WHERE post_id = ? AND comment_status = 'ACTIVE'` 형태가 반복됨. 기존 FK 기반 인덱스는 `post_id`만 포함해 후보 조회 후 `comment_status` 조건을 Filter 단계에서 추가 검사.

EXPLAIN ANALYZE로 실행 흐름 확인

EXPLAIN ANALYZE로 실제 실행 계획과 비용이 남는 위치 확인. 댓글 수 증가에 따라 Filter 단계의 불필요한 Row 검사도 함께 증가할 수 있는 구조 확인.

복합 인덱스 적용

실제 WHERE 조건에서 두 컬럼이 항상 함께 사용되는 패턴을 기준으로 `(post_id, comment_status)` 복합 인덱스 추가. 변경 후 Covering index lookup으로 실행 방식이 바뀌며 별도 Filter 단계 제거 확인.

```
CREATE INDEX idx_comment_post_id_status
ON comment(post_id, comment_status);
```

	실행 흐름
Before	Index lookup(post_id) → comment_status Filter → COUNT
After	Covering index lookup(post_id, comment_status) → COUNT

0.642ms → 0.0932ms
단일 COUNT 쿼리 실행시간

약 85.5% 감소
EXPLAIN ANALYZE 기준 실행시간 비교

운영 인프라 및 배포 구조

API 요청

모바일 앱의 요청은 `api.teenple.app → Route 53 → EC2 → Nginx :443 → Spring Boot 127.0.0.1:8080`으로 전달됩니다. Nginx에서 HTTPS를 처리하고 Spring Boot 8080은 외부에 직접 노출하지 않았습니다.

DB / Redis

영속 데이터는 Private Subnet 기반 DB Subnet Group의 RDS MySQL에 저장하고 VPC 내부 3306으로 접근합니다. Redis는 현재 EC2 내부 `127.0.0.1:6379`에서 실행합니다.

S3 / 정적 웹

게시글•채팅•프로필 미디어는 `teenple-media`, 학교 인증 학생증은 `teenple-student-card`에 저장하며 승인 후 학생증 이미지를 삭제합니다. 정적 웹은 `Route 53 → CloudFront → S3 teenple-web`로 제공합니다.

운영 접근 / 로그

EC2 Security Group은 외부에 80/443만 공개하고 22/8080/6379는 공개하지 않았습니다. 서버 접근은 SSM Session Manager를 사용하고, CloudWatch Agent가 Spring Boot와 Nginx 로그를 CloudWatch Logs로 전송합니다.

팀 CI/CD 배포 흐름



main 브랜치 push 시 GitHub Actions가 JDK 21/Gradle로 JAR를 빌드하고, OIDC로 AWS IAM Role을 Assume합니다. JAR는 S3 배포 버킷에 커밋 SHA 단위로 업로드되고, SSM SendCommand로 EC2에서 내려받아 `/opt/teenple/teenple-backend.jar`를 교체합니다. systemd 서비스 재시작 후 `/actuator/health`를 반복 확인합니다.

현재 구조에서 확인한 한계

현재는 단일 EC2, RDS Single-AZ, EC2 내부 Redis, systemd restart 배포를 사용합니다. 비용을 줄일 수 있는 대신 EC2 장애와 배포 시 짧은 중단이 발생할 수 있습니다. 트래픽과 가용성 요구가 커진다면 ALB + 다중 EC2/Auto Scaling, RDS Multi-AZ, ElastiCache, Rolling/Blue-Green 배포 등을 검토할 수 있습니다.

Project 02

배달의드론 | 가상 드론 시뮬레이터 기반 음식 배달 웹서비스

2025.03 - 2025.11 • 4인 팀 • Backend

다중 사용자 주문 처리와 외부 Python 드론 시뮬레이터 연동을 담당했습니다. 주문 트랜잭션 내부의 외부 통신 지연과 동시 주문에서 발생한 재고 정합성 문제를 부하 테스트로 재현해 개선했습니다.

주요 담당

- 다중 사용자 주문 처리 로직 구현
- 외부 시스템 연동 트랜잭션 최적화

핵심 기술

- Java · Spring Boot · Spring Data JPA · MySQL
- Python · BlockingQueue · k6

Troubleshooting 01 - 외부 Python 서버 연동 부하 분리 및 트랜잭션 최적화

문제: 여러 사용자의 동시 주문 시 일부 주문 저장 누락 및 응답 지연 발생. `@Transactional` 구간 내부의 외부 Python 서버 통신이 Blocking I/O로 동작해 대기 시간 동안 트랜잭션과 DB 커넥션 장기 점유.

해결: 트랜잭션 범위를 DB 저장 중심으로 축소하고 외부 서버 통신을 BlockingQueue 기반 비동기 파이프라인으로 분리. 주문 요청은 DB 저장을 먼저 완료하고 별도 워커 스레드가 큐의 데이터를 Python 서버로 전송하도록 변경.

결과: k6 기준 100명 동시 주문 안정적 처리. 평균 응답시간 0.7s → 0.4s, 주문 성공률 60% → 100%, TPS 약 3배 증가.

Troubleshooting 02 - 동시 주문에서의 재고 정합성과 Deadlock

문제: 동일 상품 재고에 동시 업데이트 발생, 일부 재고 차감 누락 및 주문 결과와 재고 상태 불일치.

시도: JPA 비관적 락으로 정합성 확보 후 일부 시나리오에서 Deadlock 발생. 동시 주문 반복 재현 결과, 요청별 여러 상품의 락 획득 순서가 달라 서로의 락을 대기하는 상황 확인.

해결: 상품 ID 기준으로 락 획득 순서를 고정해 모든 트랜잭션이 동일한 순서로 락을 획득하도록 변경. 재고/주문 정합성 유지, 재고 차감 누락 및 Deadlock 재발 방지.

Project 03

학사정보관리시스템 | 통합 학사 행정 관리 시스템

2025.05 - 2025.07 • 4인 팀 • Backend

전체 ERD와 수강신청•수강과목 관리 도메인을 설계했습니다. 수강신청 시간대의 동시 접속 상황을 k6로 재현하고 JPA 조회 구조와 DB 접근 횟수를 확인해 강의 목록 조회 병목을 개선했습니다.

주요 담당

- 전체 ERD 설계
- 수강신청 및 수강과목 관리 도메인 설계
- 대규모 데이터 조회 병목 개선

핵심 기술

- Java · Spring Boot · Spring Data JPA · MySQL
- Thymeleaf · k6 · Fetch Join

Troubleshooting 01 - 수강신청 강의 목록 조회 성능 개선

문제: 수강신청 페이지 강의 목록 조회에서 동시 접속 증가 시 응답 지연 급증.

측정: 500명 동시 접속•100개 과목 기준 k6 실행, 평균 응답 2,286ms • 최대 7,920ms 확인. DB 접근 횟수와 조회 쿼리 형태를 기준으로 병목 원인 분리.

결과: 조회 구조 개선 후 동일 조건에서 평균 응답시간 2,286ms → 71ms, 처리량 약 2.7배 증가.

Troubleshooting 02 - JPA 지연 로딩 N+1

문제: 강의 목록 조회 시 각 강의마다 Professor • Syllabus 추가 조회 발생, 강의 수에 비례해 SELECT 증가.

확인: 쿼리 로그에서 동일 패턴의 반복 SELECT 확인, N+1로 원인 확정. 모든 연관관계 즉시 로딩 대신 수강신청 화면에 필요한 Professor와 Syllabus만 조회 대상으로 한정.

해결: Course 조회 시 Professor와 Syllabus를 Fetch Join으로 로딩하도록 변경 후 동일 k6 조건으로 재측정. 평균 응답시간 132ms, 최대 응답시간 1.23s 수준으로 감소.